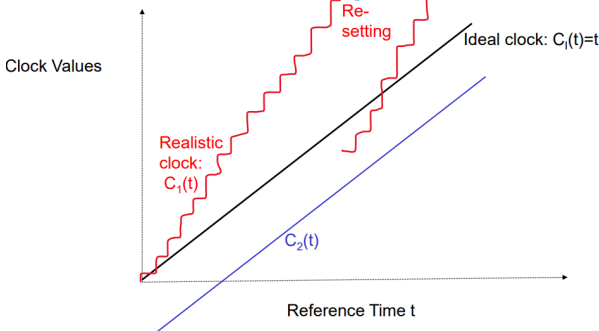


1 Clock Imperfections

Ideally every component would have a perfect clock — continuous and tracking real time exactly. In reality every clock **drifts** from true time, and no two clocks drift at the same rate.

These imperfections matter because clock offset between nodes corrupts distributed sensor averages and breaks time-based coordination.

Non-ideal clocks: Examples



We describe clock imperfections by two factors:

- **Granularity g :** (how often the clock updates)
 - **drift rate** (how fast it deviates from real time)
- Each clock has its own maximum drift rate ρ_k (Tolerance).

$$\left| \frac{dC_k(t)}{dt} - 1 \right| \leq \rho_k$$

Don't know if keep yet Modelled as cont. differentiable functions.

Importance Measures are:

Precision π : $|C_i(t) - C_k(t)| \leq \pi$ (two local clocks)

Accuracy α : $|C_k(t) - t| \leq \alpha$ (vs. UTC ref)

Because of drift, clocks must be periodically re-synced — otherwise they diverge from real time and from each other indefinitely in distributed systems.

2 Sync Protocols: Cristian's → Berkeley → NTP

Goal in distributed: all clocks agree $\Rightarrow$ minimise precision.

Sync protocols fall into two types:

- **internal** — clocks agree with each other (minimise precision, no external reference needed)
- **external** — clocks sync to a universal reference such as UTC (minimise accuracy).

We go from simplest to most sophisticated:

Cristian's Protocol

Gives the basic idea but has real-world problems;

1. Client sends request
2. Server replies: $t_B = C_B(t)$
3. Client measures RTT: $\hat{R}$
4. $C_A \leftarrow t_B + \hat{R}/2$ (precision = $\hat{R}/2$)

Problems:

- Asymm. links,
- proc. delays,
- jitter $\Rightarrow \hat{R}/2$ only estimate.

Berkeley (improves Cristian's)

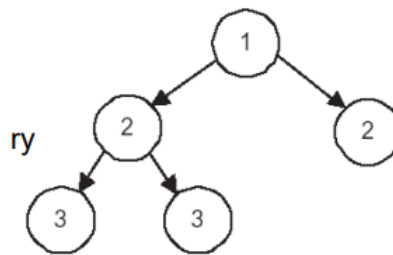
1. Master polls all slaves
2. Collects clock values
3. Estimates RTTs; removes outliers
4. Computes avg. time
5. Sends adjustments to slaves

Berkeley Introduces Filtering of outliers and polls all slaves, meaning More data points $\Rightarrow$ better estimate.

Master averages — no external UTC ref needed.

NTP — Sync to UTC via strata

NTP scales this to the internet with UTC as the reference.



Clever:

- syncs repeatedly;
- keeps offset/delay history;
- filters noisy stats;
- picks most reliable server (lowest variance).

Calculate like this:

$$\text{RTT} = (t_4 - t_1) - (t_3 - t_2)$$

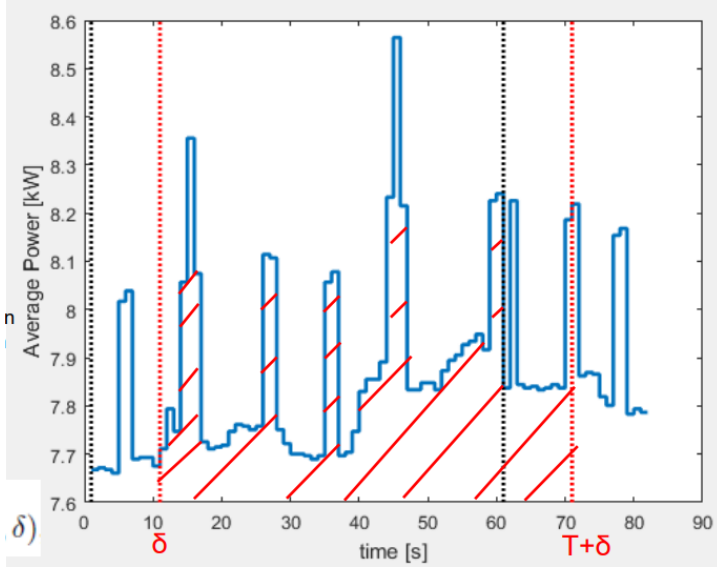
$$\text{offset} = \frac{(t_2 - t_1) + (t_3 - t_4)}{2}$$

Where **Timestamps** are:

- t_1 client send
- t_2 server rcv
- t_3 server send
- t_4 client rcv.

3 Impact of Clock Offset on Sensor Data

When distributed sensors assume clocks are synchronised and average over the same timestep (e.g. power consumption), a clock offset δ causes them to physically sample **different time intervals** — corrupting the shared average.



Even a small offset δ shifts the integration window

- two sensors integrating over period T
- averaging over physically different intervals
- results can differ significantly (e.g. $\delta=12$ ms already visible in figure).

Reference interval: $[0, T]$

Offset interval: $[\delta, T+\delta]$

Both claim same window but physically differ by δ .

Measured avg.:

$$\hat{M}(T, \delta) := \frac{1}{T} \int_{\delta}^{T+\delta} M(t) dt$$

Meas. deviation:

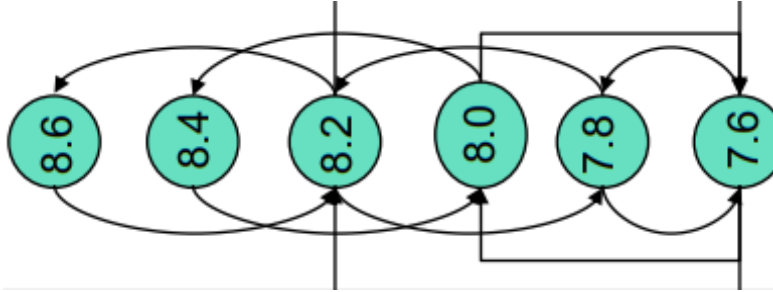
$$\varepsilon(T, \delta) := \hat{M}(T, 0) - \hat{M}(T, \delta)$$

Devices have 0.1–1% error, but *assuming* sync'd clocks is what causes the real problem — offset introduces systematic bias in averaged sensor data.

4 Markov Modulated Process

To analytically compute the measurement deviation $\varepsilon(T, \delta)$ from Section 3, we need a model for the sensor signal $M(t)$. Real signals like power consumption don't vary randomly — they switch between a few **device states** (e.g. idle, active, transmitting) that evolve as a CTMC.

The **Markov Modulated Process** extends a CTMC by attaching an emission value to each state — giving a tractable stochastic model for $M(t)$.



Extends CTMC by attaching a **value** (e.g. power) at each state. Discretise continuous meas. values to states.

State distribution:

$$\underline{P}(t) = \underline{P}(0) \exp(Qt)$$

Power emission matrix (E_{ii} = power in state i):

$$E = \text{diag}(e_1, e_2, \dots, e_n)$$

5 Expected Power & Steady-State

Steady-state distribution $\underline{\pi}$: solve $\underline{\pi}Q = \underline{0}$, $\sum_i \pi_i = 1$.

Expected power:

$$\bar{P} = \underline{\pi} \cdot \underline{e} = \sum_i \pi_i e_i$$

where $e_i = E_{ii}$ is power in state i .

For time-varying: $\underline{P}(t)E\underline{1}^T$ gives instantaneous expected power.

Clock precision $\Rightarrow$ meas. error bound:

If precision is π (two clocks differ by at most π), then offset $\delta \leq \pi$. The measurement deviation $\varepsilon(T, \delta)$ is bounded by the signal's max rate of change times δ .

Tight clock sync $\Rightarrow$ small $\delta \Rightarrow$ small $\varepsilon \Rightarrow$ valid power comparison across nodes.

Perfect vs. offset period:

$[0, T]$ vs. $[\delta, T+\delta]$ — overlap = $T-\delta$. If $\delta \ll T$ the bias is small; if $\delta \approx T$ the windows barely overlap.

6 Two Methods for Loss Estimation

L_1 : Total Loss (Energy Balance)

$$L_1 = T \cdot \left(P_{\text{imp}}^{(0)} - P_{\text{exp}}^{(0)} + \sum_{j=1}^K \left(P_{\text{inj}}^{(j)} - P_{\text{cons}}^{(j)} \right) \right)$$

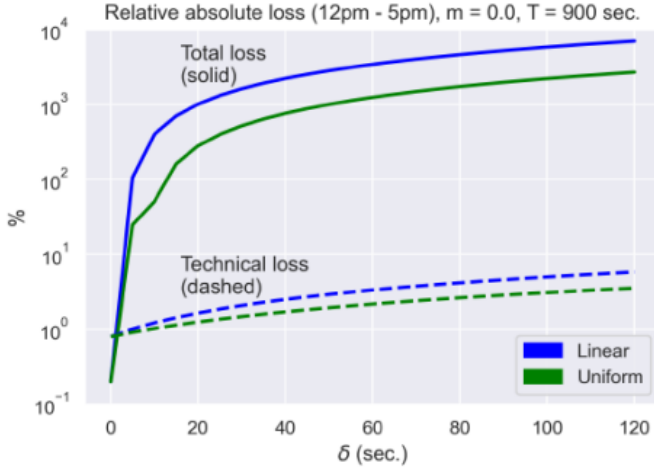
Extremely sensitive to δ : subtracts readings from *different* devices. Loss is a *small difference of large numbers*.

206–200 = 6 kW true loss; $\delta = 8$ s (machine switches on) $\Rightarrow$ one meter reads 205 kW $\Rightarrow$ computed = 1 kW. **83% error.**

L_2 : Technical Loss (Ohmic Heating)

$$L_2 = T \cdot \sum_{j=1}^C I_j^2 R_j$$

Robust to δ : each $I_j^2 R_j$ term comes from a *single sensor* on one cable segment — no cross-device subtraction. Cable current is an aggregate of many loads $\Rightarrow$ changes slowly $\Rightarrow$ edge-slice error ε stays small.



Key result (log-scale, $T = 900$ s, afternoon):

- L_1 (solid): error reaches **thousands %** by $\delta \sim 120$ s
- L_2 (dashed): saturates $\approx 10\%$ — I^2 changes smoothly

Bottom line: for NTP-synced devices ($\delta \sim \text{ms-s}$) L_1 may be ok; after reboot/GPS loss ($\delta \sim 30\text{--}120$ s) L_1 is useless. Use L_2 when clocks may drift.

Figure 5: Relative loss deviation as a function of time alignment parameter δ , afternoon trace.